Analisis Big Data

Modul Praktikum

Sains Data

Institut Teknologi Sumatera

Indeks

Alur Pelaksanaan Praktikum, 16–17

Konfigurasi Lab, 37–52
- Instalasi dan Setup Docker di WSL, 40–42
- Konfigurasi Storage Layer Menggunakan MinIO, 43–44
- Konsep Arsitektur yang Akan Diimplementasikan, 39–40
- Menambahkan Container Compute (Python) untuk Transformasi Data, 47–50
- Simulasi Ingestion Data ke Bronze Layer (Raw Data & Immutability), 45–47
- Spesifikasi Lingkungan, 37–38
- Transformasi Silver ke Gold Layer (Agregasi dan Optimasi Analitik), 50–52

Konsep Dasar, 19–37, 56–57, 64–66, 68–70, 72–75, 78–81, 84–87, 90
- Algoritma Machine Learning, 91–93
- Apache Airflow, 79
- Apache Atlas, 80
- Apache Kafka: Distributed Messaging System, 85
- Apache Mahout, 94
- Arsitektur Big Data, 25–29
- Arsitektur Hadoop, 56
- Containerization, 34–36
- Data Lake dan Object Storage, 29–30
- Data Lineage dan Impact Analysis, 80
- Data Partitioning dalam Kafka, 85

Data Skew, 69
DataFrame API Lanjutan, 64
Delivery Semantics, 85
Evaluasi Model, 93
Evolusi dari Batch ke Streaming, 84
Evolusi Pengelolaan Data, 19–23
Evolusi Struktur Data di Big Data, 72
Fault Tolerance dalam Streaming, 86–87
Feature Engineering dalam Big Data, 93
File Optimization, 69
Format Data untuk Analitik, 73
Hadoop Ecosystem Overview, 57
HBase sebagai NoSQL Distributed Database, 74
HDFS (Hadoop Distributed File System), 56
Hive sebagai SQL Layer di atas Hadoop, 72–73
Integrasi Airflow dengan Spark dan Hive, 79
Integrasi End-to-End, 75
Integrasi End-to-End Architecture, 81
Integrasi Hive dengan Spark, 73
Integrasi Kafka dengan Spark, 86
Integrasi ML dengan Pipeline Big Data, 93–94
Integrasi Storage dengan Compute, 36–37
Join dan Relational Processing, 64
Karakteristik Big Data, 23–25
Konsep Dasar Machine Learning, 90–

91
Konsep Data Governance, 79–80
Konsep Data Pipeline dalam Big Data, 78
Konsep Data Warehouse di atas Data Lake, 73–74
Konsep Event: Driven Architecture, 84
MapReduce (Secara Konseptual), 57
Medallion Architecture, 30–33
Medallion Implementation (Engineering View), 66
Metadata Management, 80
ML Pipeline Concept, 91
Observability dan Monitoring, 69–70
Optimisasi Dasar, 68
Orkestrasi Workflow, 78–79
Partisi Data, 65
Peran Machine Learning dalam Ekosistem Big Data, 90
Perbandingan Hive vs HBase, 74
Pipeline Medallion End-to-End, 69
Prinsip Desain Data Lake, 33–34
Spark Execution Plan, 68
Spark MLlib Architecture, 91
Storage Strategy dalam Big Data, 74–75
Strategi Penyimpanan Data, 65
Stream Processing Model, 86
Transformasi Wide vs Narrow, 65
Windowing Concept, 86
YARN (Yet Another Resource Negotiator), 57
Kriteria Penilaian, 15–16

Latihan, 53–54
Latihan Terpandu: Implementasi Pipeline Menggunakan World Happiness Report, 53–54

Panduan Asisten Praktikum, 11–13
Tugas Paska Praktikum, 12–13
Tugas Saat Praktikum, 11–12

Panduan untuk Praktikan (Mahasiswa), 13–15
Ketentuan Disiplin dan Etika Akademik, 15
Ketentuan Umum, 14
Pelaksanaan Praktikum, 14
Pengumpulan dan Dokumentasi, 15
Penutupan Praktikum, 15
Persiapan Sebelum Praktikum, 14

Tujuan, 19, 55, 59–62,63, 67, 71–72, 77–78, 83–84, 89–90
Arsitektur Apache Spark, 60–61
Evolusi dari MapReduce ke Spark, 60
Format Data dalam Spark, 62
Integrasi Spark dengan Storage, 61
Konsep RDD (Resilient Distributed Dataset), 61
Spark Execution Model, 61
Spark UI dan Monitoring, 62

Daftar Isi

Indeks **3**

Panduan Praktikum **11**
1 Panduan Asisten Praktikum . . . 11
1.1 Tugas Saat Praktikum . . . 11
1.2 Tugas Paska Praktikum . . . 12
2 Panduan untuk Praktikan (Mahasiswa) . . . 13
2.1 Ketentuan Umum . . . 14
2.2 Persiapan Sebelum Praktikum . . . 14
2.3 Pelaksanaan Praktikum . . . 14
2.4 Pengumpulan dan Dokumentasi . . . 15
2.5 Ketentuan Disiplin dan Etika Akademik . . . 15
2.6 Penutupan Praktikum . . . 15
3 Kriteria Penilaian . . . 15
4 Alur Pelaksanaan Praktikum . . . 16

Modul 1: Desain Arsitektur dan Platform Big Data **19**
1 Tujuan . . . 19
2 Konsep Dasar . . . 19
2.1 Evolusi Pengelolaan Data . . . 19
Database Relasional . . . 20
Data Warehouse . . . 21
Data Lake . . . 22
Lakehouse: Konvergensi Dua Dunia . . . 22
2.2 Karakteristik Big Data . . . 23
Model 5V . . . 23
Batch vs. Streaming Processing . . . 24
Klasifikasi Berdasarkan Struktur Data . . . 25
2.3 Arsitektur Big Data . . . 25
2.4 Data Lake dan Object Storage . . . 29
Apa Itu Object Storage? . . . 29
File System vs. Object Storage . . . 29
S3 API dan Konsep Dasarnya . . . 30
Perbandingan dengan HDFS . . . 30
2.5 Medallion Architecture . . . 30
Zona Bronze — Raw dan Immutable . . . 31
Zona Silver — Cleaned dan Standardized . . . 31
Zona Gold — Aggregated dan Business-Ready . . . 31

Alur Transformasi dan Data Lineage 32
2.6 Prinsip Desain Data Lake 33
Immutability 33
Schema-on-Read 33
Data Partitioning 33
Metadata Management 34
2.7 Containerization 34
Apa Itu Container? 34
VM vs. Container 35
Konsep Dasar Docker 36
2.8 Integrasi Storage dengan Compute 36
Bagaimana Spark Membaca dari Object Storage 36
3 Konfigurasi Lab 37
3.1 Spesifikasi Lingkungan 37
3.2 Konsep Arsitektur yang Akan Diimplementasikan 39
1. Prinsip Pemisahan Storage dan Compute 39
2. Implementasi Data Lake dengan Medallion Architecture 39
3. Arsitektur Berbasis Container 40
4. Alur Data yang Akan Disimulasikan 40
3.3 Instalasi dan Setup Docker di WSL 40
1. Prasyarat Sistem 41
2. Instalasi Docker pada WSL 41
3. Konfigurasi Hak Akses Docker 42
4. Verifikasi Instalasi 42
5. Konfigurasi Resource (Opsional) 42
3.4 Konfigurasi Storage Layer Menggunakan MinIO 43
1. Pembuatan Struktur Project 43
2. Konfigurasi Docker Compose 43
3. Menjalankan MinIO 44
4. Pembuatan Bucket Medallion 44
3.5 Simulasi Ingestion Data ke Bronze Layer (Raw Data & Immutability) 45
1. Konsep Bronze Layer 45
2. Menyiapkan Dataset Mentah 45
3. Mengunggah Data ke Bucket Bronze 46
4. Prinsip Immutability 46
5. Verifikasi Ingestion 47
3.6 Menambahkan Container Compute (Python) untuk Transformasi Data 47
1. Menambahkan Service Python pada Docker Compose 47
2. Membuat Folder Aplikasi Compute 48
3. Install Dependency di Container Compute 49
4. Simulasi Alur Data 49
5. Makna Arsitektural 49

3.7 Transformasi Silver ke Gold Layer (Agregasi dan Optimasi Analitik) . . . 50
1. Konsep Gold Layer . . . 50
2. Menambahkan Script Agregasi . . . 50
3. Menjalankan Proses Agregasi . . . 51
4. Alur Arsitektur Lengkap . . . 52
5. Interpretasi Hasil . . . 52
4 Latihan . . . 53
4.1 Latihan Terpandu: Implementasi Pipeline Menggunakan World Happiness Report . . . 53
Langkah 1: Upload ke Bronze . . . 53
Langkah 2: Transformasi ke Silver . . . 53
Langkah 3: Agregasi ke Gold . . . 54
Analisis Hasil . . . 54

Modul 2: Ekosistem Hadoop **55**
1 Tujuan . . . 55
2 Konsep Dasar . . . 56
2.1 Arsitektur Hadoop . . . 56
2.2 HDFS (Hadoop Distributed File System) . . . 56
2.3 YARN (Yet Another Resource Negotiator) . . . 57
2.4 MapReduce (Secara Konseptual) . . . 57
2.5 Hadoop Ecosystem Overview . . . 57

Modul 3: Pemrosesan Big Data Bagian 1 **59**
1 Tujuan . . . 59
2 Tujuan . . . 60
2.1 Evolusi dari MapReduce ke Spark . . . 60
2.2 Arsitektur Apache Spark . . . 60
2.3 Konsep RDD (Resilient Distributed Dataset) . . . 61
2.4 Spark Execution Model . . . 61
2.5 Integrasi Spark dengan Storage . . . 61
2.6 Format Data dalam Spark . . . 62
2.7 Spark UI dan Monitoring . . . 62

Modul 4: Pemrosesan Big Data Bagian 2 **63**
1 Tujuan . . . 63
2 Konsep Dasar . . . 64
2.1 DataFrame API Lanjutan . . . 64
2.2 Join dan Relational Processing . . . 64
2.3 Transformasi Wide vs Narrow . . . 65
2.4 Partisi Data . . . 65
2.5 Strategi Penyimpanan Data . . . 65
2.6 Medallion Implementation (Engineering View) . . . 66

Modul 5: Pemrosesan Big Data Bagian 3 **67**
1 Tujuan 67
2 Konsep Dasar 68
2.1 Spark Execution Plan 68
2.2 Optimisasi Dasar 68
2.3 Data Skew 69
2.4 File Optimization 69
2.5 Pipeline Medallion End-to-End 69
2.6 Observability dan Monitoring 69

Modul 6: Struktur dan Penyimpanan Big Data **71**
1 Tujuan 71
2 Konsep Dasar 72
2.1 Evolusi Struktur Data di Big Data 72
2.2 Hive sebagai SQL Layer di atas Hadoop 72
2.3 Integrasi Hive dengan Spark 73
2.4 Format Data untuk Analitik 73
2.5 Konsep Data Warehouse di atas Data Lake 73
2.6 HBase sebagai NoSQL Distributed Database 74
2.7 Perbandingan Hive vs HBase 74
2.8 Storage Strategy dalam Big Data 74
2.9 Integrasi End-to-End 75

Modul 7: Orkestrasi Alur Kerja dan Tata Kelola Big Data **77**
1 Tujuan 77
2 Konsep Dasar 78
2.1 Konsep Data Pipeline dalam Big Data 78
2.2 Orkestrasi Workflow 78
2.3 Apache Airflow 79
2.4 Integrasi Airflow dengan Spark dan Hive 79
2.5 Konsep Data Governance 79
2.6 Metadata Management 80
2.7 Apache Atlas 80
2.8 Data Lineage dan Impact Analysis 80
2.9 Integrasi End-to-End Architecture 81

Modul 8: Analitik Aliran Data **83**
1 Tujuan 83
2 Konsep Dasar 84
2.1 Evolusi dari Batch ke Streaming 84
2.2 Konsep Event: Driven Architecture 84
2.3 Apache Kafka: Distributed Messaging System 85
2.4 Data Partitioning dalam Kafka 85
2.5 Delivery Semantics 85
2.6 Stream Processing Model 86

2.7 Windowing Concept 86
2.8 Integrasi Kafka dengan Spark 86
2.9 Fault Tolerance dalam Streaming 86

Modul 9: Machine Learning Big Data **89**
1 Tujuan 89
2 Konsep Dasar 90
2.1 Peran Machine Learning dalam Ekosistem Big Data 90
2.2 Konsep Dasar Machine Learning 90
2.3 Spark MLlib Architecture 91
2.4 ML Pipeline Concept 91
2.5 Algoritma Machine Learning 91
Linear Regression 92
Logistic Regression 92
Decision Tree 92
K-Means clustering 93
2.6 Feature Engineering dalam Big Data 93
2.7 Evaluasi Model 93
2.8 Integrasi ML dengan Pipeline Big Data 93
2.9 Apache Mahout 94

Panduan Praktikum

1 Panduan Asisten Praktikum

Asisten praktikum adalah individu yang bertugas membantu dosen atau instruktur dalam menyelenggarakan dan membimbing mahasiswa selama sesi praktikum. Peran asisten praktikum mencakup berbagai aspek teknis dan akademik, termasuk menjelaskan konsep-konsep materi, memberikan bimbingan dalam penggunaan perangkat lunak atau alat analisis, serta mengevaluasi tugas dan laporan praktikum mahasiswa. Sebagai asisten praktikum Analisis Big Data, keterampilan dalam pemrograman, pemahaman tentang teknologi Big Data, serta kemampuan komunikasi dan pembimbingan yang baik sangat diperlukan. Beberapa tugas utama asisten praktikum meliputi:

1.1 Tugas Saat Praktikum

Beberapa tugas utama asisten praktikum adalah sebagai berikut:

1. Mempersiapkan lingkungan praktikum sebelum sesi dimulai, termasuk memastikan seluruh perangkat lunak, container, dan layanan (misalnya Docker, MinIO, Hadoop, Spark) berjalan dengan baik dan dapat diakses oleh mahasiswa.

2. Melakukan pengecekan kesiapan infrastruktur laboratorium, seperti koneksi jaringan, spesifikasi perangkat, serta konfigurasi resource (CPU, RAM, storage) yang dibutuhkan untuk menjalankan ekosistem Big Data.

3. Memberikan penjelasan teknis tambahan terkait materi praktikum, khususnya pada bagian implementasi arsitektur, konfigurasi sistem, serta alur pemrosesan data.

4. Membimbing mahasiswa dalam proses instalasi, konfigurasi, dan troubleshooting apabila terjadi kesalahan pada saat deployment container atau eksekusi job.

5. Mengawasi pelaksanaan praktikum agar sesuai dengan modul dan capaian pembelajaran yang telah ditetapkan.

6. Membantu mahasiswa memahami perbedaan antara konsep teoretis dan implementasi teknis, seperti pemisahan storage layer dan compute layer, arsitektur medallion (bronze, silver, gold), serta integrasi antar komponen sistem Big Data.
7. Memberikan contoh penggunaan perintah dasar (command-line) yang relevan, seperti manajemen container, manajemen file pada object storage atau HDFS, serta proses submit job Spark.
8. Mengidentifikasi dan mendokumentasikan kendala teknis yang sering muncul selama praktikum untuk dijadikan bahan evaluasi dan perbaikan modul di masa mendatang.
9. Membantu proses evaluasi tugas praktikum dengan melakukan verifikasi terhadap hasil implementasi mahasiswa, baik dari sisi struktur arsitektur, kebenaran pemrosesan data, maupun kualitas dokumentasi laporan.
10. Memberikan umpan balik konstruktif terhadap laporan praktikum mahasiswa, terutama pada aspek desain arsitektur, konsistensi implementasi, efisiensi pemrosesan data, serta pemahaman konsep Big Data.
11. Menjaga suasana akademik yang kondusif selama praktikum berlangsung serta memastikan komunikasi antara dosen dan mahasiswa berjalan efektif.

1.2 Tugas Paska Praktikum

Adapun tugas paska praktikum adalah tugas yang dibebankan kepada mahasiswa untuk mengerjakan permasalahan yang diberikan setelah selesai melaksanakan praktikum di laboratorium. Tugas yang diberikan wajib dikerjakan secara individual dalam kurun waktu yang telah disepakati oleh mahasiswa dan asisten praktikum. Beberapa tugas asisten praktikum setelah praktikum adalah sebagai berikut:

1. **Menilai Tugas dan Laporan**
   Pada tahap pasca-praktikum, asisten bertanggung jawab melakukan penilaian terhadap tugas dan laporan yang dikumpulkan oleh mahasiswa. Proses ini mencakup pemeriksaan ketepatan penyelesaian, kesesuaian dengan instruksi praktikum, serta kualitas penyajian hasil kerja.

Penilaian dilakukan secara objektif berdasarkan kriteria atau rubrik yang telah ditetapkan agar hasil evaluasi adil dan terukur.

2. **Memberikan Umpan Balik Konstruktif**
   Selain memberikan nilai, asisten juga perlu menyampaikan umpan balik yang bersifat membangun. Umpan balik ini bertujuan membantu mahasiswa memahami kelebihan dan kekurangan dari pekerjaan yang telah dilakukan, serta memberikan arahan untuk perbaikan di masa mendatang. Penyampaian dilakukan secara jelas, spesifik, dan edukatif agar dapat meningkatkan kualitas pemahaman dan keterampilan mahasiswa.

3. **Menyusun Rekap Evaluasi**
   Asisten juga bertugas menyusun rekapitulasi hasil evaluasi sebagai bahan refleksi dan laporan kepada dosen pengampu. Rekap ini dapat mencakup distribusi nilai, pola kesalahan yang sering terjadi, serta tingkat pencapaian kompetensi mahasiswa. Informasi tersebut berguna untuk menilai efektivitas pelaksanaan praktikum dan menjadi dasar perbaikan pada pertemuan berikutnya.

2 Panduan untuk Praktikan (Mahasiswa)

Praktikum Analisis Big Data dilaksanakan 1 kali dalam satu minggu atau hanya satu kali pertemuan setiap modul. Lamanya durasi pertemuan praktikum adalah 120 menit. Berikut jumlah modul dan topik praktikum setiap modul dan ujian praktikum:

1. Modul 1: Desain Arsitektur dan Platform Big Data
2. Modul 2: Ekosistem Hadoop
3. Modul 3: Pemrosesan Big Data Bagian 1
4. Modul 4: Pemrosesan Big Data Bagian 2
5. Modul 5: Pemrosesan Big Data Bagian 3
6. Modul 6: Struktur dan Penyimpanan Big Data
7. Modul 7: Orkestrasi Alur Kerja dan Tata Kelola Big Data
8. Modul 8: Analitik Aliran Data
9. Modul 9: Machine Learning Big Data
10. Ujian Praktikum

2.1 Ketentuan Umum

1. Mahasiswa wajib hadir tepat waktu sesuai jadwal praktikum yang telah ditentukan.
2. Mahasiswa harus mengenakan pakaian yang sopan dan sesuai dengan ketentuan laboratorium.
3. Setiap peserta wajib menjaga ketertiban, kebersihan, dan keamanan selama berada di dalam laboratorium.
4. Dilarang membawa makanan dan minuman ke dalam ruang laboratorium.
5. Mahasiswa wajib mematuhi seluruh instruksi dosen maupun asisten praktikum.

2.2 Persiapan Sebelum Praktikum

1. Mahasiswa diwajibkan mempelajari modul praktikum sebelum sesi dimulai.
2. Mahasiswa harus memastikan akun, perangkat lunak, atau kebutuhan teknis lainnya telah siap digunakan.
3. Keterlambatan melebihi batas toleransi yang ditetapkan dapat mengakibatkan tidak diperkenankan mengikuti praktikum pada sesi tersebut.

2.3 Pelaksanaan Praktikum

1. Praktikum diawali dengan penjelasan singkat mengenai tujuan dan materi yang akan dikerjakan.
2. Mahasiswa mengerjakan tugas sesuai instruksi pada modul dan arahan asisten.
3. Setiap mahasiswa wajib mengerjakan tugas secara mandiri kecuali ditentukan sebagai kerja kelompok.
4. Apabila mengalami kendala teknis atau kesalahan program, mahasiswa diharapkan mencoba menganalisis terlebih dahulu sebelum meminta bantuan.
5. Penggunaan perangkat laboratorium harus sesuai prosedur dan tidak diperkenankan mengubah konfigurasi sistem tanpa izin.

2.4 Pengumpulan dan Dokumentasi

1. Tugas praktikum dikumpulkan sesuai mekanisme yang telah ditentukan (misalnya melalui LMS atau repository).
2. Setiap laporan harus disusun secara sistematis dan mengikuti format yang telah ditetapkan.
3. Keterlambatan pengumpulan dapat dikenakan pengurangan nilai sesuai kebijakan.

2.5 Ketentuan Disiplin dan Etika Akademik

1. Segala bentuk plagiarisme atau kecurangan akademik tidak diperkenankan dan akan dikenakan sanksi sesuai peraturan yang berlaku.
2. Mahasiswa wajib menjaga integritas akademik dalam seluruh proses praktikum.
3. Kerusakan fasilitas laboratorium akibat kelalaian menjadi tanggung jawab pihak yang bersangkutan.

2.6 Penutupan Praktikum

1. Mahasiswa wajib memastikan perangkat dalam keadaan baik sebelum meninggalkan laboratorium.
2. Seluruh file pekerjaan harus disimpan dengan benar untuk menghindari kehilangan data.
3. Laboratorium harus ditinggalkan dalam kondisi tertib dan bersih.

3 Kriteria Penilaian

Adapun kriteria penilaian dalam pelaksanaan praktikum ini adalah ditunjukkan pada Tabel 1.

Penilaian yang dilakukan per modul adalah:

1. Pre-Test
2. Tugas Individu di Lab
3. Tugas Take Home

Tabel 1: *Kriteria Penilaian*

Komponen	Bobot
Pre-test	10 %
Tugas Lab	30 %
Tugas Take Home	30 %
Ujian Praktikum	30 %

Nilai ini dicatat setiap pertemuan.

Setelah 9 modul selesai, nilai dihitung sebagai berikut:

1. Pre-Test (Total 10%)
   Rumus:
   $Nilai\ PreTest = \frac{PT_1+PT_2+...+PT_9}{9}$ Lalu dikalikan 10 % .

2. Tugas Individu di Lab (Total 30 %)
   Rumus:
   $Nilai\ Lab = \frac{T_1+T_2+...+T_9}{9}$ Lalu dikalikan 30 % .

3. Tugas Take Home (Total 30 %)
   Rumus:
   $Nilai\ TakeHome = \frac{TH_1+TH_2+...+TH_9}{9}$ Lalu dikalikan 30 % .

lalu total nilai ujian praktikum adalah 30 %. Maka nilai akhir praktikum adalah sebagai berikut:
$Nilai\ Akhir = (PreTest \times 10\%) + (Lab \times 30\%) + (TakeHome \times 30\%) + (Ujian \times 30\%)$

4 Alur Pelaksanaan Praktikum

Praktikum dilaksanakan dalam beberapa tahap yaitu:

1. Pre-Test
   Mahasiswa mengikuti pre-test yang dilaksanakan sebelum memulai kegiatan percobaan, asisten praktikum akan memberikan soal pre-test secara langsung dan waktu pengerjaannya adalah 10 menit. Asisten praktikum mencatat hasil dan membagikan hasil pre-test ketika akhir pertemuan.

2. Kegiatan Percobaan/Penjelasan Asisten Praktikum
   Percobaan dilakukan mengikuti prosedur percobaan yang ditunjukkan di modul praktikum. Asisten praktikum

menjelaskan modul yang akan mahasiswa kerjakan, memandu secara teknis hingga penjelasan analitiknya. Penjelasan oleh asisten praktikum berlangsung maksimal dalam kurun waktu 10 menit.

3. Kegiatan Tugas Kolaboratif atau Tugas Lab
   Setelah berhasil menyimak penjelasan asisten praktikum, mahasiswa mengerjakan dan menjawab pertanyaan atau mengikuti instruksi dari modul dan asisten praktikum pada tugas kolaboratif. Output tugas kolaboratif adalah berhasil membuat script code dan output yang sesuai dengan instruksi dalam modul. Kegiatan ini dilaksanakan 90 menit, setelah selesai mengerjakan tugas kolaboratif, mahasiswa wajib mengumpulkan ke asisten praktikum melalui link yang disediakan dan dilarang untuk meninggalkan laboratorium sebelum pengarahan tugas individu oleh asisten praktikum. Tugas dikumpulkan langsung di hari praktikum.

4. Kegiatan Membuat Laporan Individu
   Asisten praktikum memberikan pengarahan dan tanya jawab terkait Tugas Individu yang sesuai dengan pembagian oleh asisten praktikum dalam kurun waktu 10 menit. Diskusi dapat berupa instruksi, petunjuk pengerjaan, dan waktu bimbingan/asistensi secara kolektif. Tugas individu dikumpulkan H+3 hari setelah praktikum.

Modul 1: Desain Arsitektur dan Platform Big Data

1 Tujuan

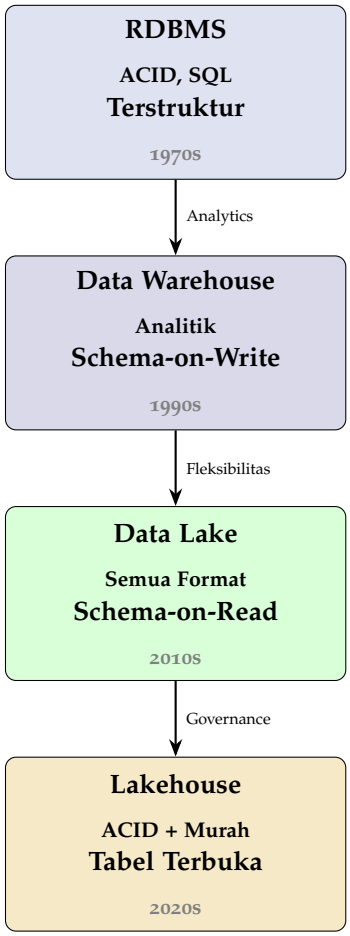
Modul 1 bertujuan untuk memberikan pemahaman konseptual dan implementatif mengenai desain arsitektur Big Data berbasis *data lakehouse* dengan pendekatan *medallion architecture*. Pada akhir praktikum ini, mahasiswa diharapkan mampu:

1. Menjelaskan perbedaan antara *Data Warehouse*, *Data Lake*, dan *Lakehouse* dalam konteks sistem Big Data modern.
2. Memahami konsep pemisahan *storage layer* dan *compute layer* sebagai fondasi arsitektur Big Data terdistribusi.
3. Mendesain struktur zona data (*bronze*, *silver*, dan *gold*) beserta alur pemrosesan data yang sistematis dan terstandarisasi.
4. Mengimplementasikan lingkungan *object storage* berbasis container menggunakan Docker dan MinIO sebagai simulasi *data lake*.
5. Mengelola data mentah (*raw data*) ke dalam zona *bronze* serta memahami prinsip *immutability* dan partisi data.

2 Konsep Dasar

2.1 Evolusi Pengelolaan Data

Sebelum membahas arsitektur modern, penting untuk memahami mengapa lanskap pengelolaan data terus berevolusi. Setiap generasi teknologi muncul sebagai respons terhadap keterbatasan generasi sebelumnya, dan memahami keterbatasan tersebut adalah kunci untuk mengerti desain sistem big data masa kini.



Gambar 1: *Evolusi paradigma pengelolaan data dari waktu ke waktu*

Database Relasional

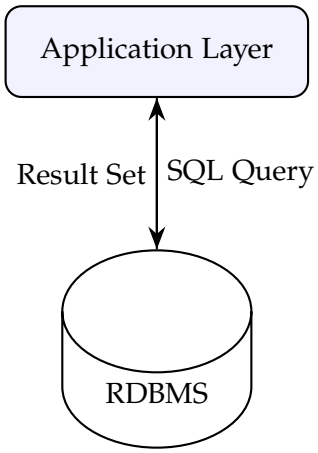
Database relasional (RDBMS) muncul pada era 1970-an sebagai solusi pengelolaan data terstruktur yang andal. Dengan menggunakan model tabel dan bahasa SQL, RDBMS seperti Oracle, MySQL, dan PostgreSQL menjadi tulang punggung hampir semua aplikasi bisnis selama beberapa dekade.

Kekuatan utama RDBMS terletak pada konsistensi data yang dijamin oleh properti ACID (*Atomicity, Consistency, Isolation, Durability*), kemampuan melakukan query ad-hoc yang fleksibel, dan dukungan transaksi yang matang. Namun ketika volume data mulai meledak di era internet, keterbatasan RDBMS mulai tampak jelas.

Skalabilitas RDBMS bersifat vertikal — untuk menangani beban lebih besar, server harus di-upgrade (lebih banyak CPU, RAM). Pendekatan ini mahal dan memiliki batas fisik. Selain itu, RDBMS tidak dirancang untuk menyimpan data tidak

terstruktur seperti gambar, teks bebas, atau log JSON.

Secara arsitektural, RDBMS dirancang untuk beban kerja OLTP (Online Transaction Processing). Fokusnya adalah konsistensi dan integritas transaksi, bukan pemrosesan data skala besar atau analitik kompleks lintas jutaan baris.



Gambar 2: *Database Relasional*

Data Warehouse

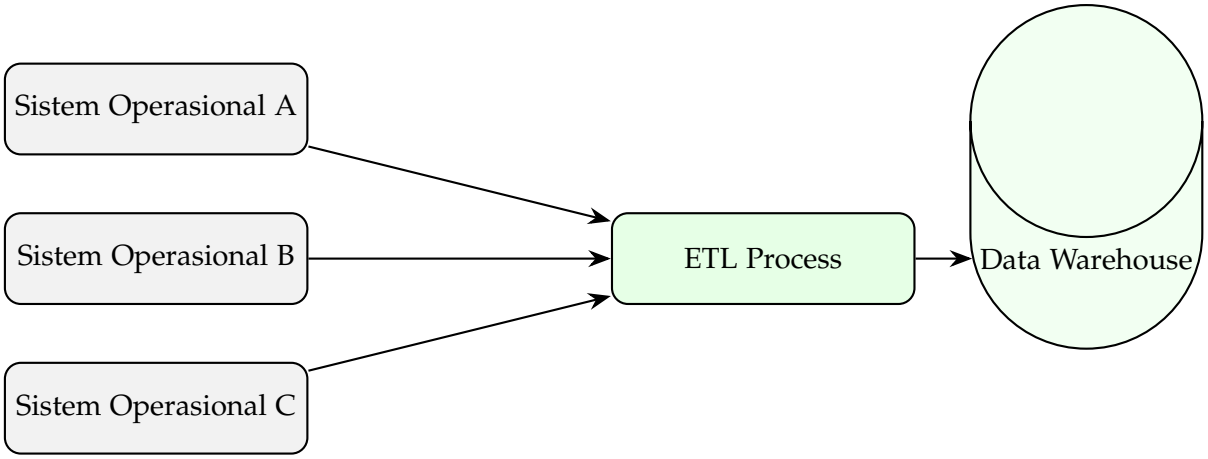
Data Warehouse (DWH) hadir sebagai solusi untuk kebutuhan analitik bisnis yang tidak bisa dipenuhi oleh RDBMS operasional. Konsep ini dipopulerkan oleh Bill Inmon dan Ralph Kimball pada awal 1990-an, dengan ide bahwa data dari berbagai sistem operasional perlu diintegrasikan, dibersihkan, dan disimpan dalam satu repositori terpusat untuk keperluan pelaporan dan analisis.

DWH menggunakan skema khusus seperti *star schema* atau *snowflake schema* yang dioptimalkan untuk query analitik. Proses ETL (*Extract, Transform, Load*) menjadi tulang punggung pengisian data ke dalam warehouse. Solusi seperti Teradata, IBM Netezza, dan kemudian Amazon Redshift serta Google BigQuery mendominasi pasar ini.

Permasalahan muncul ketika kebutuhan data semakin beragam. DWH mensyaratkan struktur data yang sudah ditentukan sebelumnya (*schema-on-write*), sehingga setiap perubahan kebutuhan bisnis memerlukan perubahan skema yang mahal dan memakan waktu. Data yang tidak sesuai skema terpaksa dibuang atau diabaikan.

Data Warehouse dioptimalkan untuk OLAP (Online Analytical Processing). Struktur seperti star schema meminimalkan

join kompleks dan mempercepat agregasi multidimensi.



Gambar 3: *Data Warehouse*

Data Lake

Data Lake muncul sekitar tahun 2010 sebagai jawaban atas keterbatasan DWH. Prinsip utamanya adalah menyimpan semua data mentah — terstruktur, semi-terstruktur, maupun tidak terstruktur — tanpa perlu mendefinisikan skema terlebih dahulu. Filosofi ini dikenal sebagai *schema-on-read*: struktur data baru diterapkan saat data dibaca, bukan saat disimpan.

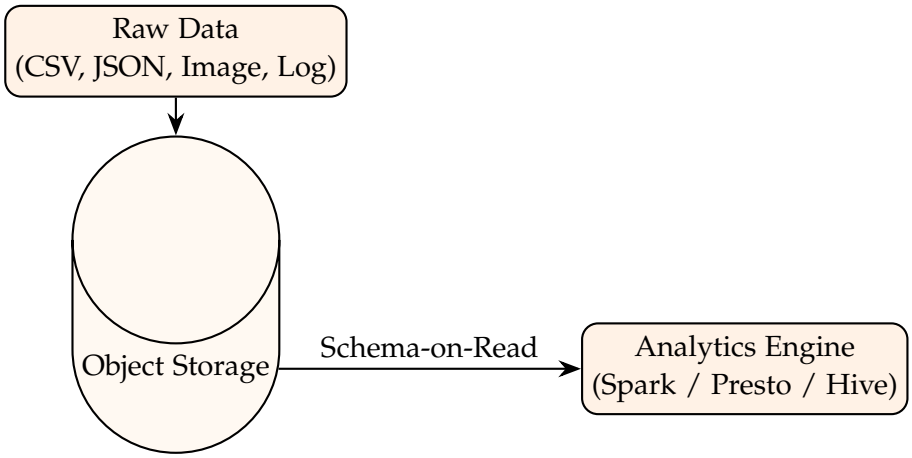
Dengan menggunakan sistem file terdistribusi seperti HDFS atau object storage seperti Amazon S3, Data Lake dapat menampung data dalam skala petabyte dengan biaya yang jauh lebih rendah. Semua data disimpan dalam format aslinya, memberikan fleksibilitas maksimal bagi para ilmuwan data untuk bereksperimen.

Namun dalam praktiknya, banyak Data Lake yang berubah menjadi apa yang disebut *data swamp* — kubangan data yang kacau dan tidak terkelola. Tanpa governance yang baik, data lake penuh dengan data duplikat, metadata yang hilang, dan kualitas data yang tidak terjamin.

Fleksibilitas ini memungkinkan eksplorasi data secara eksperimental, namun tanpa metadata management dan data catalog, lake dengan cepat kehilangan nilai bisnisnya.

Lakehouse: Konvergensi Dua Dunia

Lakehouse adalah paradigma arsitektur yang menggabungkan kelebihan Data Warehouse (performa query tinggi, dukung-



Gambar 4: *Data Lake*

an ACID, manajemen skema) dengan fleksibilitas Data Lake (penyimpanan murah, dukungan berbagai format, skalabilitas masif). Konsep ini dipopulerkan oleh Databricks melalui Delta Lake, dan kemudian diikuti oleh Apache Iceberg dan Apache Hudi.

Kunci inovasi Lakehouse adalah lapisan metadata transaksional di atas object storage biasa. Lapisan ini memungkinkan operasi DML (`UPDATE`, `DELETE`, `MERGE`) yang sebelumnya tidak tersedia di Data Lake, serta menyediakan *time travel* untuk mengakses versi data historis.

Lakehouse bukan sekadar tren, melainkan konvergensi alami dari dua jalur evolusi yang berbeda. Pemahaman tentang mengapa setiap generasi muncul adalah fondasi untuk merancang sistem big data yang tepat sasaran.

Lakehouse memperkenalkan konsep table abstraction di atas object storage. Data tetap disimpan sebagai file (Parquet/ORC), namun diperlakukan sebagai tabel transaksional yang mendukung ACID dan versioning.

2.2 Karakteristik Big Data

Model 5V

Big Data bukan hanya soal ukuran data yang besar. Komunitas industri dan akademik mendefinisikan big data melalui lima dimensi karakteristik yang dikenal sebagai **5V**. Memahami 5V secara konseptual sangat penting sebelum memilih arsitektur dan teknologi yang tepat.

Volume merujuk pada skala data yang dihasilkan. Sebuah platform media sosial besar dapat menghasilkan ratusan terabyte data per hari. Skala ini melampaui kapasitas RDBMS konvensional dan menuntut sistem penyimpanan terdistribusi.

Velocity merujuk pada kecepatan generasi dan pemrosesan data. Data dari sensor IoT, transaksi keuangan, dan klik pengguna dihasilkan secara terus-menerus. Sistem yang tidak mampu memproses data dengan kecepatan sebanding akan mengalami penundaan (*lag*) yang dapat memengaruhi keputusan bisnis.

Variety merujuk pada keberagaman format dan sumber data. Data modern tidak lagi hanya berupa tabel terstruktur, tetapi mencakup JSON dari API, log teks, gambar, audio, video, dan data sensor biner. Tantangannya adalah mengintegrasikan semua format ini dalam satu platform analitik.

Veracity merujuk pada kualitas dan kepercayaan data. Data yang besar namun penuh noise, duplikasi, dan inkonsistensi justru berbahaya karena dapat menghasilkan kesimpulan yang salah. Dimensi ini mendorong pentingnya validasi dan pembersihan data.

Value adalah tujuan akhir dari seluruh upaya big data — mengekstrak nilai bisnis yang berarti dari data yang ada. Volume besar tidak ada gunanya jika tidak menghasilkan wawasan yang dapat ditindaklanjuti.

Batch vs. Streaming Processing

Cara data diproses sangat bergantung pada sifat use case bisnisnya. Terdapat dua paradigma utama pemrosesan data yang perlu dipahami.

Batch Processing memproses data dalam kelompok besar pada interval waktu tertentu — misalnya setiap malam atau setiap minggu. Pendekatan ini cocok untuk laporan bisnis periodik, rekonsiliasi keuangan, dan pelatihan model machine learning. MapReduce dan Spark dalam mode batch adalah representasi paling dikenal dari paradigma ini.

Stream Processing memproses data segera setelah data tersebut dihasilkan, dengan latensi dalam hitungan milidetik hingga detik. Ini diperlukan untuk deteksi fraud real-time, monitoring sistem, dan personalisasi konten langsung. Apache Kafka dikombinasikan dengan Spark Structured Streaming atau Apache Flink adalah solusi umum untuk kebutuhan ini.

Pada banyak sistem modern, kedua paradigma ini dikombinasikan dalam apa yang disebut *Lambda Architecture* atau *Kappa*

Architecture, di mana jalur batch dan streaming bekerja paralel atau saling melengkapi.

Klasifikasi Berdasarkan Struktur Data

Data dapat diklasifikasikan berdasarkan derajat strukturnya:

- **Terstruktur** — Data yang mengikuti skema ketat dengan tipe dan panjang kolom yang terdefinisi. Contoh: tabel database relasional, spreadsheet Excel.
- **Semi-terstruktur** — Data yang memiliki struktur fleksibel namun mengandung penanda metadata. Contoh: JSON, XML, YAML, log dengan field tertentu.
- **Tidak terstruktur** — Data tanpa skema yang dapat diidentifikasi secara otomatis. Contoh: teks bebas, gambar, audio, video, dokumen PDF.

Sistem big data modern harus mampu mengelola ketiga kategori ini secara efisien dalam satu platform, yang menjadi salah satu alasan mengapa Data Lake dengan object storage menjadi fondasi arsitektur yang populer.

2.3 Arsitektur Big Data

Arsitektur sistem big data modern menggunakan pendekatan berlapis (*layered architecture*) yang memisahkan tanggung jawab setiap komponen secara jelas. Pemisahan ini memungkinkan setiap lapisan untuk dikembangkan, diskalakan, dan diganti secara independen tanpa memengaruhi lapisan lain.

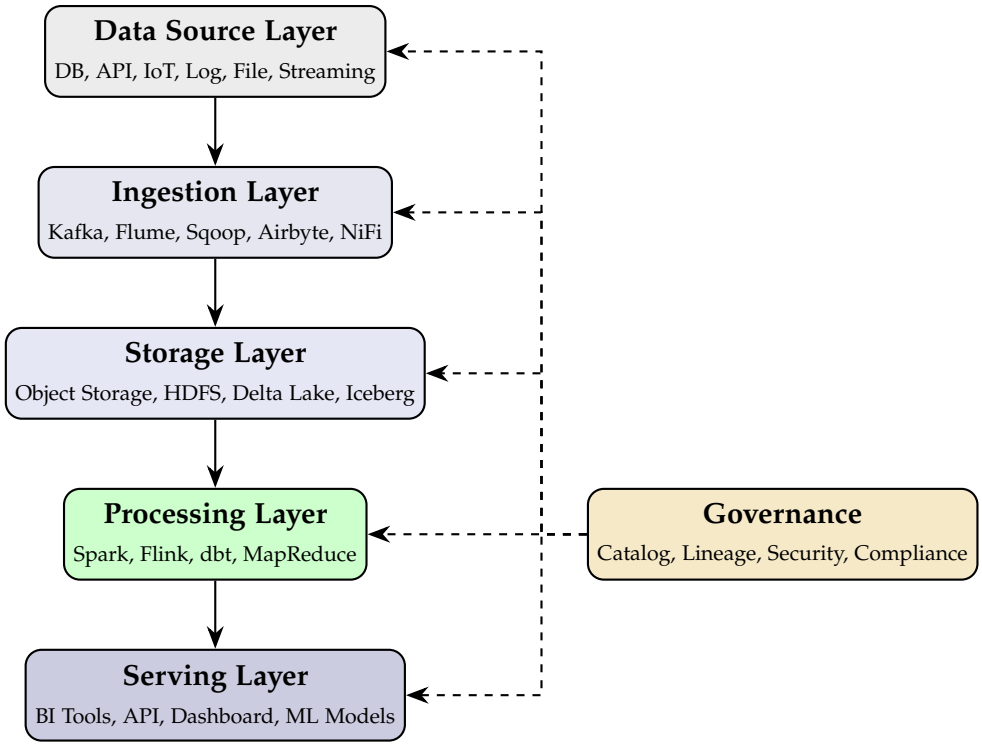
Arsitektur sistem big data modern umumnya mengadopsi pendekatan *layered architecture* yang memisahkan tanggung jawab setiap komponen secara eksplisit. Pemisahan ini bukan sekadar desain teknis, melainkan prinsip fundamental untuk mencapai **skalabilitas, fault tolerance, dan evolvabilitas sistem**.

Setiap lapisan memiliki kontrak fungsional yang jelas. Perubahan pada satu lapisan—misalnya mengganti engine pemrosesan dari Spark ke Flink—tidak semestinya memaksa perubahan pada lapisan penyimpanan. Inilah konsep **decoupling** yang menjadi inti arsitektur data modern.

Secara konseptual, arsitektur ini dapat dibagi menjadi dua domain besar:

1. **Data Plane**: aliran data aktual dari sumber hingga konsumsi.

2. **Control Plane**: metadata, keamanan, dan governance yang mengontrol bagaimana data digunakan.



Gambar 5: *Arsitektur berlapis sistem Big Data modern*

Data Source Layer adalah titik asal semua data — database operasional, API eksternal, sensor IoT, file log, media sosial, dan sebagainya. Lapisan ini tidak dikendalikan oleh platform big data itu sendiri, tetapi menjadi sumber yang perlu diadaptasi.

Lapisan ini adalah titik asal seluruh data. Sumber data dapat berupa:

- Sistem OLTP (RDBMS operasional)
- API eksternal
- Sensor IoT
- Log aplikasi
- Streaming event

Karakteristik utama lapisan ini adalah heterogenitas dan ketidakpastian struktur. Sistem big data tidak mengontrol sumber, melainkan harus beradaptasi terhadap variasinya.

Ingestion Layer bertanggung jawab mengumpulkan data dari berbagai sumber dan meneruskannya ke sistem penyimpanan. Untuk data batch, tools seperti Sqoop atau Airbyte digunakan. Untuk streaming, Apache Kafka berperan sebagai *message broker* terdistribusi yang menjamin pengiriman data dengan throughput sangat tinggi.

Lapisan ini bertugas melakukan akuisisi data secara:

1. Batch ingestion: data dikumpulkan secara periodik.
2. Streaming ingestion: data diproses secara real-time.

Apache Kafka, misalnya, berperan sebagai *distributed commit log* yang menjamin ordering dan durability event dalam throughput sangat tinggi. Ingestion adalah titik kritis untuk menjamin:

- Idempotensi
- Reliability
- Back-pressure handling

Tanpa ingestion yang stabil, seluruh pipeline menjadi rapuh.

Storage Layer adalah lapisan yang menyimpan data dalam berbagai tahap pemrosesan. Inilah tempat MinIO (atau S3) berperan — sebagai object storage terdistribusi yang menjadi fondasi Data Lake.

Lapisan penyimpanan dalam arsitektur modern biasanya berbasis object storage (MinIO, S3, GCS) yang bersifat:

1. Immutable
2. Highly scalable
3. Cost-efficient

Berbeda dengan database tradisional, storage layer ini tidak mengikat compute engine tertentu. Data disimpan dalam format terbuka seperti Parquet atau ORC. Inilah fondasi dari konsep **separation of storage and compute**.

Processing Layer berisi engine komputasi yang mentransformasi data dari satu bentuk ke bentuk lain. Apache Spark adalah representasi paling dominan dari lapisan ini, mampu memproses data dalam skala ratusan terabyte menggunakan komputasi terdistribusi.

Lapisan ini melakukan:

1. Transformasi data
2. Enrichment
3. Aggregation
4. Feature engineering (untuk ML)

Apache Spark menjadi representasi dominan karena mendukung:

1. Batch processing
2. Structured streaming
3. SQL engine
4. Machine learning pipeline

Processing layer bekerja di atas storage tanpa mengubah struktur fisik penyimpanan.

Serving Layer menyajikan data yang sudah diproses ke pengguna akhir dalam berbagai bentuk: dashboard BI, REST API untuk aplikasi, atau data mart untuk tim analitik. Serving layer bertanggung jawab menyajikan data yang sudah terkurasi ke pengguna akhir dalam bentuk: Dashboard BI, REST API, Data mart, Model ML inference, Lapisan ini harus mengoptimalkan latensi dan ketersediaan.

Governance Layer bukan lapisan fisik tersendiri, melainkan kapabilitas horizontal yang mencakup semua lapisan lain. Governance mengatur siapa yang boleh mengakses data apa, bagaimana data dilacak lineage-nya, dan bagaimana kepatuhan regulasi dipastikan. Governance bukan lapisan vertikal, melainkan kapabilitas horizontal yang mencakup:

1. Data catalog
2. Lineage tracking
3. Access control
4. Audit & compliance

Tanpa governance, sistem big data cenderung berubah menjadi data swamp yang tidak terkontrol.

Salah satu prinsip terpenting dalam arsitektur big data modern adalah pemisahan storage dan compute. Berbeda dengan database tradisional di mana penyimpanan dan pemrosesan terikat erat dalam satu server, arsitektur modern memungkinkan keduanya diskalakan secara independen. Ini adalah alasan

Tabel 1: *Perbandingan File System POSIX dengan Object Storage*

Aspek	File System (POSIX)	Object Storage (S3)
Struktur	Hierarki direktori	Flat namespace
Akses	Path absolut	Bucket dan Key
Metadata	Terbatas (OS-level)	Kaya dan kustom
Skalabilitas	Vertikal (terbatas)	Horizontal (sangat besar)
Konsistensi	Strong consistency	Eventual/Strong (tergantung impl.)
Biaya	Mahal per GB	Murah per GB
Modifikasi	In-place edit	Objek baru (immutable)

mengapa kita menggunakan MinIO sebagai storage terpisah sebelum menghubungkannya dengan Spark sebagai compute engine.

2.4 Data Lake dan Object Storage

Apa Itu Object Storage?

Object storage adalah paradigma penyimpanan data yang memperlakukan setiap data sebagai sebuah *objek* yang berdiri sendiri, lengkap dengan data itu sendiri, metadata deskriptif, dan sebuah *identifier* unik. Berbeda dengan file system tradisional yang mengorganisir data dalam hierarki direktori, object storage menggunakan namespace datar (*flat namespace*) yang jauh lebih mudah untuk diskalakan.

Ketika kita menyimpan sebuah file CSV berukuran 500 MB ke object storage, file tersebut disimpan sebagai satu objek dengan metadata seperti ukuran, tipe konten, tanggal modifikasi, dan metadata kustom yang dapat kita definisikan. Objek ini diakses melalui identifier unik, bukan melalui path direktori.

File System vs. Object Storage

Perbedaan mendasar antara file system hierarkis dan object storage perlu dipahami sebelum merancang Data Lake.

S3 API dan Konsep Dasarnya

Amazon S3 (Simple Storage Service) tidak hanya merupakan produk cloud, tetapi juga telah menjadi **standar de facto API** untuk object storage. MinIO, Ceph, dan berbagai penyedia cloud lainnya mengimplementasikan S3 API yang sama, sehingga aplikasi yang ditulis untuk S3 dapat berjalan di atas storage mana pun yang kompatibel dengan S3.

Tiga konsep paling fundamental dalam S3 API adalah:

- **Bucket** — Kontainer tingkat teratas untuk menyimpan objek. Nama bucket harus unik secara global. Analogikan sebagai drive atau partisi utama.
- **Object** — Unit data terkecil yang disimpan, dapat berupa file apa pun — CSV, Parquet, JSON, gambar, dll. Setiap objek memiliki metadata dan identifier.
- **Key** — Identifier unik sebuah objek dalam sebuah bucket. Meskipun key sering menggunakan karakter `/` yang terlihat seperti path direktori (misalnya `data/bronze/2024/01/events.parquet`), ini hanyalah nama string — tidak ada hierarki direktori yang sebenarnya.

Perbandingan dengan HDFS

HDFS (*Hadoop Distributed File System*) adalah pendahulu object storage dalam ekosistem big data. HDFS menyimpan data dalam blok berukuran 128 MB yang didistribusikan ke banyak *DataNode*, dengan *NameNode* sebagai metadata manager terpusat.

Meskipun HDFS sangat andal untuk workload Hadoop, ia memiliki beberapa keterbatasan dibanding object storage modern: NameNode yang terpusat menjadi *single point of failure* potensial, biaya operasional klaster HDFS lebih tinggi, dan HDFS tidak mendukung akses HTTP/REST natively. Object storage S3-compatible menawarkan semua fungsionalitas serupa dengan biaya lebih rendah, kemudahan operasional, dan integrasi ekosistem yang lebih luas.

2.5 Medallion Architecture

Medallion Architecture adalah pola desain Data Lake yang mengorganisir data dalam tiga zona atau lapisan kualitas yang progresif. Pola ini dipopulerkan oleh Databricks dan telah

menjadi praktik terbaik industri dalam merancang Data Lake yang dapat dipelihara dan dapat dipercaya.

Zona Bronze — Raw dan Immutable

Zona Bronze adalah titik masuk pertama semua data ke dalam Data Lake. Di sini, data disimpan **persis** seperti saat diterima dari sumbernya — tanpa transformasi, tanpa pembersihan, tanpa filtering. Prinsip paling kritis di zona ini adalah **immutability**: data yang sudah masuk tidak boleh diubah atau dihapus.

Filosofi immutability di Bronze memiliki alasan yang kuat. Pertama, data mentah bertindak sebagai *source of truth* yang dapat selalu dirujuk kembali. Kedua, jika proses transformasi menghasilkan output yang salah, kita selalu dapat mengulang dari data mentah. Ketiga, data mentah sering memiliki nilai audit dan compliance yang penting.

Contoh isi zona Bronze: file CSV yang langsung di-upload dari sistem ERP, JSON dari webhook API pihak ketiga, atau log biner dari perangkat IoT. Format penyimpanan biasanya mengikuti format asli sumber data.

Zona Silver — Cleaned dan Standardized

Zona Silver berisi data yang sudah melalui proses pembersihan dan standardisasi dari Bronze. Di sini, duplikat dihapus, tipe data dikonversi, nilai null ditangani, dan skema distandarisasi. Data di Silver sudah dapat digunakan untuk analitik dasar, tetapi belum diagregasi untuk kebutuhan bisnis spesifik.

Transformasi yang terjadi dari Bronze ke Silver meliputi: deduplikasi berdasarkan primary key, normalisasi format tanggal dan waktu, penerapan validasi bisnis dasar (misalnya harga tidak boleh negatif), penggabungan data dari beberapa sumber, dan konversi format dari CSV ke Parquet untuk efisiensi query.

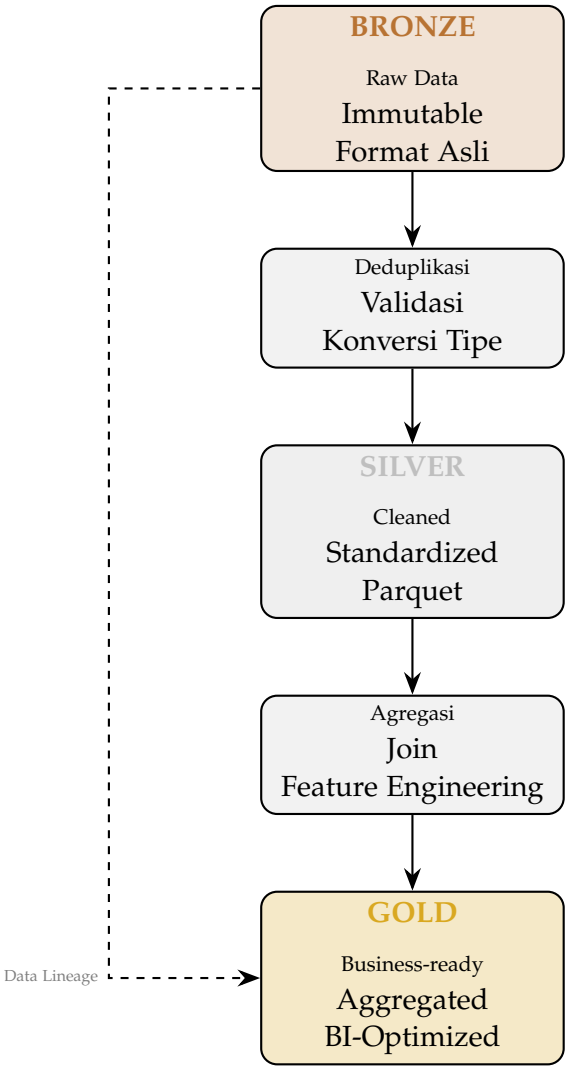
Zona Gold — Aggregated dan Business-Ready

Zona Gold berisi data yang sudah diagregasi dan dibentuk sesuai kebutuhan bisnis spesifik. Data di zona ini adalah yang paling siap untuk dikonsumsi oleh dashboard BI, laporan eksekutif, atau model machine learning produksi.

Contoh data di zona Gold: tabel penjualan bulanan per wilayah, metrik retensi pelanggan, atau fitur yang sudah di-engineer untuk model prediksi. Data Gold biasanya memiliki granula-

ritas yang lebih rendah (sudah diagregasi) namun sangat teroptimasi untuk kasus penggunaan tertentu.

Alur Transformasi dan Data Lineage



Gambar 6: *Alur transformasi data dalam Medallion Architecture*

Data Lineage merujuk pada kemampuan untuk menelusuri asal-usul sebuah data — dari mana datangnya, melalui transformasi apa, dan ke mana perginya. Dalam Medallion Architecture, lineage secara natural terbentuk karena setiap zona mewakili satu tahap transformasi yang terdokumentasi. Ini sangat penting untuk debugging ketika ditemukan anomali di zona Gold, karena kita dapat menelusuri kembali ke Bronze untuk menemukan sumber masalah.

Medallion Architecture merepresentasikan strategi pemurnian data bertahap (progressive refinement). Bronze layer menyimpan data mentah sebagai single source of truth yang immutable. Silver layer memastikan kualitas dan konsistensi skema melalui standardisasi. Gold layer menyajikan data yang telah dioptimalkan untuk kebutuhan bisnis dan analitik.

Pendekatan ini memungkinkan isolasi kompleksitas transformasi serta mendukung reproducibility dan data lineage yang eksplisit — prinsip penting dalam sistem Lakehouse modern.

2.6 Prinsip Desain Data Lake

Immutability

Immutability berarti data yang sudah tersimpan tidak boleh dimodifikasi secara in-place. Setiap perubahan menghasilkan versi baru dari data tersebut. Prinsip ini memiliki beberapa manfaat penting: memungkinkan time-travel query (melihat data pada titik waktu tertentu), menyederhanakan recovery dari kesalahan, dan memudahkan audit trail untuk kepatuhan regulasi.

Object storage secara natural mendukung immutability karena model penulisannya adalah *write-once* — untuk "mengubah" sebuah objek, kita sebenarnya menulis objek baru dengan key yang sama, dan versi lama dapat dipertahankan melalui fitur versioning.

Schema-on-Read

Berbeda dengan database relasional yang menggunakan *schema-on-write* (skema harus didefinisikan sebelum data masuk), Data Lake menggunakan *schema-on-read*: data disimpan tanpa skema tetap, dan skema baru diterapkan saat data dibaca untuk keperluan analitik. Ini memberikan fleksibilitas untuk mengingesti data dari sumber baru tanpa perlu mengubah infrastruktur penyimpanan, dan memungkinkan tim yang berbeda untuk menginterpretasikan data yang sama dengan skema yang berbeda sesuai kebutuhan mereka.

Data Partitioning

Partitioning adalah teknik mengorganisir data dalam object storage menggunakan hierarki key yang mencerminkan dimensi yang sering digunakan untuk filtering. Misalnya, data event dapat dipartisi berdasarkan tanggal:

Tabel 2: *Perbandingan format penyimpanan CSV dan Parquet*

Aspek	CSV (Row-based)	Parquet (Columnar)
Penyimpanan	Baris demi baris	Kolom demi kolom
Kompresi	Rendah	Sangat tinggi (4–10x)
Query kolom tertentu	Baca semua kolom	Baca kolom relevan saja
Skema	Tidak ada	Embedded dalam file
Ekosistem	Universal	Spark, Hive, Presto, dll.
Ideal untuk	Transfer data sederhana	Analitik skala besar

`events/year=2024/month=01/day=15/data.parquet`. Ketika query hanya membutuhkan data bulan Januari 2024, mesin query dapat langsung melompat ke partisi yang relevan tanpa membaca seluruh dataset — teknik ini dikenal sebagai *partition pruning* dan secara dramatis meningkatkan performa query.

Format kolumnar seperti Parquet menyimpan data per kolom, bukan per baris. Ketika sebuah query analitik hanya membutuhkan kolom `revenue` dan `date` dari sebuah tabel dengan 100 kolom, Parquet hanya perlu membaca 2 kolom tersebut. Ini menghasilkan penghematan I/O yang sangat signifikan, apalagi dikombinasikan dengan kompresi algoritma Snappy atau ZSTD yang efisien untuk data kolumnar.

Metadata Management

Metadata adalah data tentang data. Dalam skala Data Lake, tanpa metadata yang baik, menemukan dataset yang relevan menjadi seperti mencari jarum dalam tumpukan jerami. Metadata mencakup informasi seperti skema data, statistik distribusi nilai, informasi pemilik data, tanggal pembaruan terakhir, dan hubungan lineage. Tools seperti Apache Atlas, AWS Glue Data Catalog, atau Unity Catalog (Databricks) menyediakan kemampuan katalog metadata yang terpusat.

2.7 Containerization

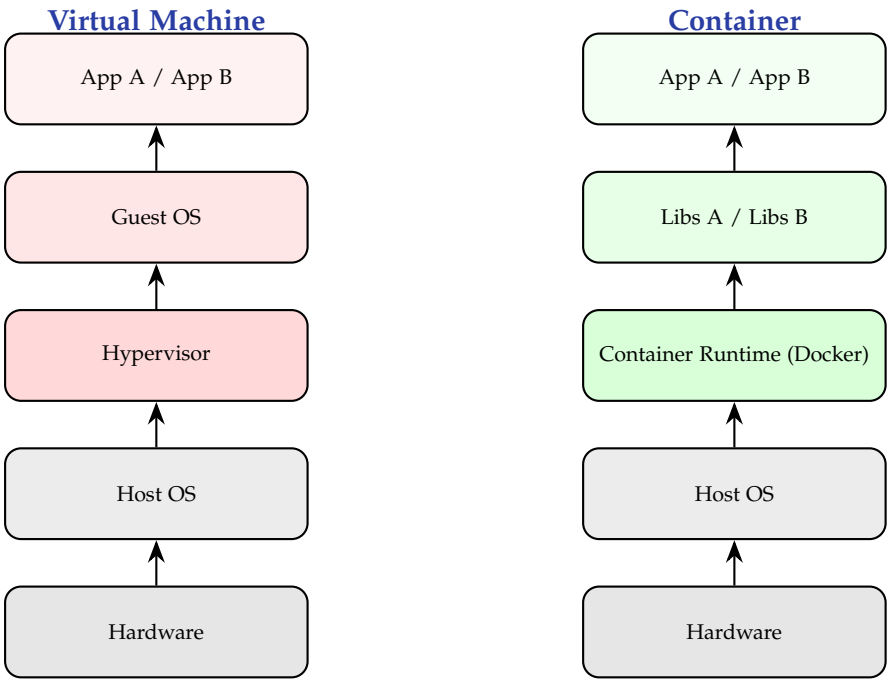
Apa Itu Container?

Container adalah unit deployment yang mengemas aplikasi beserta seluruh dependensinya — kode, runtime, library, konfigurasi — dalam satu paket yang dapat berjalan secara kon-

sisten di lingkungan mana pun. Docker adalah platform containerization yang paling banyak digunakan, dan pemahaman tentang container sangat penting karena hampir semua laboratorium dalam mata kuliah ini menggunakan Docker.

Cara paling mudah memahami container adalah dengan analogi kapal kargo. Sebelum era kontainer pengiriman, setiap barang dikemas secara berbeda-beda dan memerlukan penanganan khusus. Container standarisasi proses ini: barang apa pun yang masuk ke dalam kontainer dapat diangkut kapal mana pun. Demikian pula, container Docker memungkinkan aplikasi apa pun berjalan di atas infrastruktur mana pun tanpa masalah kompatibilitas.

VM vs. Container



Gambar 7: *Perbandingan arsitektur Virtual Machine dan Container*

Virtual Machine mengemulasi seluruh perangkat keras, sehingga setiap VM menjalankan sistem operasi tamu yang lengkap. Ini menghasilkan isolasi yang sangat kuat namun dengan overhead yang signifikan — setiap VM membutuhkan ratusan MB RAM hanya untuk OS-nya.

Container berbagi kernel OS host, sehingga jauh lebih ringan. Sebuah container dapat di-start dalam hitungan detik, tidak perlu booting OS, dan mengonsumsi memori hanya untuk aplikasi dan library-nya saja. Pada lab big data, ini sangat

penting karena kita perlu menjalankan beberapa komponen (MinIO, Spark, Jupyter) secara bersamaan dalam satu mesin lokal.

Konsep Dasar Docker

Tiga konsep dasar Docker yang perlu dipahami adalah:

- **Image** — Template read-only yang mendefinisikan isi sebuah container. Image dibangun dari `Dockerfile` dan dapat di-share melalui registry seperti Docker Hub. Image untuk MinIO, misalnya, berisi seluruh binari MinIO dan konfigurasi dasarnya.
- **Container** — Instance yang berjalan dari sebuah image. Dari satu image yang sama, kita dapat menjalankan banyak container secara bersamaan.
- **Network** — Mekanisme yang memungkinkan container-container untuk berkomunikasi satu sama lain. Docker Compose membuat jaringan virtual di mana semua service dapat saling mengakses menggunakan nama service sebagai hostname.

Containerization menjawab masalah klasik "works on my machine" — konfigurasi yang tidak konsisten antar lingkungan pengembangan. Dengan Docker Compose, seluruh stack lab dapat di-replikasi dengan perintah `docker-compose up` tanpa instalasi manual apapun. Hal ini memastikan setiap mahasiswa bekerja dalam lingkungan yang identik.

2.8 Integrasi Storage dengan Compute

Modul 1 berfokus pada pemahaman dan penggunaan MinIO sebagai object storage. Namun penting untuk melihat ke depan: bagaimana storage ini akan digunakan bersama compute engine seperti Apache Spark di modul-modul berikutnya.

Bagaimana Spark Membaca dari Object Storage

Apache Spark tidak membaca data secara langsung dari disk lokal, melainkan melalui konektor (*connector*) yang mengimplementasikan protokol akses storage. Untuk membaca dari S3-compatible storage seperti MinIO, Spark menggunakan

library `hadoop-aws` yang mengimplementasikan S3A filesystem — sebuah adapter yang menerjemahkan operasi file Spark menjadi API call S3.

Dari perspektif kode Spark, membaca dari MinIO tidak berbeda jauh dengan membaca file lokal, hanya dengan skema URL yang berbeda:

```
df = spark.read.parquet("s3a://data-lake/bronze/events/")
```

Di balik layar, Spark akan melakukan serangkaian panggilan S3 API untuk mendaftar objek dalam prefix tersebut, membaca metadata Parquet dari setiap file, dan mendistribusikan pembacaan data ke seluruh executor dalam klaster.

3 Konfigurasi Lab

3.1 Spesifikasi Lingkungan

Modul 1 dijalankan menggunakan lingkungan *containerized* berbasis Docker dengan dukungan *Windows Subsystem for Linux* (WSL2). Pendekatan ini memungkinkan simulasi arsitektur Big Data secara lokal namun tetap merepresentasikan prinsip *cloud-native infrastructure*.

1. Sistem Operasi

- Windows 10/11 (64-bit)
- WSL2 terinstal dan aktif
- Distribusi Linux yang direkomendasikan: Ubuntu 22.04 LTS

Verifikasi WSL:

```
wsl --status
wsl --list --verbose
```

Pastikan versi WSL adalah **Version 2**.

2. Docker Engine

Docker digunakan sebagai platform orkestrasi container untuk menjalankan MinIO (Object Storage).

- Docker Desktop for Windows (WSL2 backend aktif)
- Docker Engine minimal versi 24.x
- Docker Compose plugin aktif

3. Spesifikasi Perangkat Keras Minimum

Agar simulasi berjalan stabil, disarankan spesifikasi berikut:

- RAM minimal 8 GB (direkomendasikan 16 GB)
- Storage kosong minimal 10 GB
- CPU dual-core atau lebih

Alokasi resource Docker Desktop (Settings ke Resources):

- Memory: 4–6 GB
- CPU: 2 core atau lebih

4. Port yang Digunakan

Praktikum ini menggunakan beberapa port lokal:

- 9000 : MinIO API (S3-compatible endpoint)
- 9001 : MinIO Web Console

Pastikan port tersebut tidak digunakan oleh aplikasi lain.

6. Prinsip Lingkungan Praktikum

Lingkungan yang dibangun mengikuti prinsip:

1. **Containerization** — setiap komponen berjalan dalam container terisolasi.
2. **Separation of Storage and Compute** — MinIO sebagai storage layer berdiri terpisah dari engine komputasi.
3. **Reproducibility** — seluruh konfigurasi dapat direplikasi menggunakan `docker-compose.yml`.
4. **Local Cloud Simulation** — arsitektur lokal meniru pola arsitektur cloud (S3-compatible object storage).

Dengan lingkungan ini, mahasiswa akan memahami bahwa arsitektur Big Data modern tidak bergantung pada satu server monolitik, melainkan pada komponen terdistribusi yang dapat diskalakan secara independen.

3.2 Konsep Arsitektur yang Akan Diimplementasikan

Pada praktikum Modul 1 ini, arsitektur yang akan dibangun mengadopsi prinsip **pemisahan storage dan compute** dalam sistem Big Data modern. Lingkungan ini dirancang untuk mensimulasikan *Data Lake* sederhana menggunakan **object storage berbasis MinIO** yang dijalankan di dalam container Docker pada lingkungan WSL.

1. Prinsip Pemisahan Storage dan Compute

Arsitektur Big Data modern memisahkan:

- **Storage Layer** : Bertanggung jawab menyimpan data secara persisten.
- **Compute Layer** : Bertanggung jawab melakukan pemrosesan dan transformasi data.

Dalam praktikum ini:

- **Storage Layer** diimplementasikan menggunakan MinIO (object storage kompatibel S3).
- **Compute Layer** disimulasikan menggunakan container terpisah (misalnya Python container atau CLI tools).

Pendekatan ini mencerminkan pola arsitektur sistem terdistribusi seperti AWS S3 + Spark, GCS + Dataproc, atau Azure Data Lake + Synapse.

2. Implementasi Data Lake dengan Medallion Architecture

Data akan diorganisasikan ke dalam tiga zona utama:

1. **Bronze Layer** Berisi data mentah (*raw data*) yang bersifat *immutable*. Tidak dilakukan modifikasi terhadap data asli.
2. **Silver Layer** Berisi data yang telah melalui proses:
   - Pembersihan data
   - Validasi skema
   - Standarisasi format
3. **Gold Layer** Berisi data yang telah:

- Diagregasi
- Dioptimalkan untuk analitik
- Siap dikonsumsi sistem BI atau dashboard

Struktur ini akan direpresentasikan dalam bucket MinIO sebagai berikut:

```
bigdata-lab/
|-- bronze/
|-- silver/
`-- gold/
```

3. Arsitektur Berbasis Container

Seluruh komponen akan dijalankan menggunakan Docker Compose:

- Container MinIO : Object Storage
- Container Client / Processing : Simulasi Compute

4. Alur Data yang Akan Disimulasikan

Alur pemrosesan data pada praktikum ini adalah sebagai berikut:

1. Dataset mentah diunggah ke zona **bronze**.
2. Data dibaca oleh container compute.
3. Dilakukan transformasi dan validasi.
4. Hasil disimpan ke zona **silver**.
5. Dilakukan agregasi sederhana.
6. Output akhir disimpan di zona **gold**.

Melalui arsitektur ini, mahasiswa tidak hanya memahami konsep teoritis Data Lake dan Medallion Architecture, tetapi juga melihat implementasi praktis pemisahan storage dan compute dalam lingkungan terkontainerisasi.

3.3 Instalasi dan Setup Docker di WSL

Pada praktikum ini, seluruh simulasi arsitektur Big Data akan dijalankan menggunakan Docker di dalam lingkungan **Windows Subsystem for Linux (WSL2)**. WSL memungkinkan pengguna Windows menjalankan kernel Linux secara native tanpa virtual machine terpisah.

1. Prasyarat Sistem

Pastikan sistem memenuhi spesifikasi berikut:

- Windows 10/11 (64-bit) dengan dukungan virtualisasi aktif
- WSL2 terinstal
- Distribusi Linux (disarankan: Ubuntu 22.04)
- Minimal RAM 8 GB
- Minimal ruang kosong 10 GB

Untuk memverifikasi versi WSL:

```
wsl --list --verbose
```

Pastikan versi yang digunakan adalah **Version 2**.

2. Instalasi Docker pada WSL

Update sistem terlebih dahulu:

```
sudo apt update && sudo apt upgrade -y
```

Install dependency dasar:

```
sudo apt install ca-certificates curl gnupg lsb-release -y
```

Tambahkan GPG key resmi Docker:

```
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
```

Tambahkan repository Docker:

```
echo \
"deb [arch=$(dpkg --print-architecture) \
signed-by=/etc/apt/keyrings/docker.gpg] \
https://download.docker.com/linux/ubuntu \
$(lsb_release -cs) stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Install Docker Engine dan Docker Compose plugin:

```
sudo apt update
sudo apt install docker-ce docker-ce-cli \
containerd.io docker-buildx-plugin docker-compose-plugin -y
```

3. Konfigurasi Hak Akses Docker

Agar Docker dapat dijalankan tanpa `sudo`, tambahkan user ke group docker:

```
sudo usermod -aG docker $USER
```

Keluar dari sesi WSL dan masuk kembali, atau jalankan:

```
newgrp docker
```

4. Verifikasi Instalasi

Uji apakah Docker berjalan dengan benar:

```
docker --version
docker compose version
```

Lakukan uji container sederhana:

```
docker run hello-world
```

Jika berhasil, sistem akan menampilkan pesan konfirmasi bahwa Docker telah berjalan dengan benar.

5. Konfigurasi Resource (Opsional)

Untuk memastikan performa optimal, atur alokasi resource WSL dengan membuat file:

```
C:\Users\<username>\.wslconfig
```

Isi dengan konfigurasi berikut:

```
[wsl2]
memory=6GB
processors=4
```

Restart WSL dengan perintah:

```
wsl --shutdown
```

Pengaturan ini penting agar simulasi object storage dan pemrosesan data berjalan stabil.

3.4 Konfigurasi Storage Layer Menggunakan MinIO

Pada tahap ini, kita akan membangun *object storage* berbasis MinIO menggunakan Docker Compose. MinIO berfungsi sebagai simulasi **Data Lake** yang kompatibel dengan protokol Amazon S3.

1. Pembuatan Struktur Project

Buat direktori kerja untuk praktikum:

```
mkdir -p ~/bigdata-lab-1
cd ~/bigdata-lab-1
```

Buat struktur folder berikut:

```
bigdata-lab-1/
|-- docker-compose.yml
|-- data/
|-- raw-data/
```

Folder `data/` akan digunakan sebagai volume penyimpanan MinIO.

2. Konfigurasi Docker Compose

Buat file `docker-compose.yml`:

```
nano docker-compose.yml
```

Isi dengan konfigurasi berikut:

```
version: '3.8'

services:
  minio:
    image: minio/minio:latest
    container_name: bigdata-minio
    ports:
      - "9000:9000"
      - "9001:9001"
    environment:
      MINIO_ROOT_USER: admin
      MINIO_ROOT_PASSWORD: admin123
    volumes:
      - ./data:/data
    command: server /data --console-address ":9001"
```

Penjelasan konfigurasi:

- Port 9000 : Endpoint S3 API
- Port 9001 : Web Console MinIO
- Volume `./data:/data` : Penyimpanan persisten
- `MINIO_ROOT_USER` dan `MINIO_ROOT_PASSWORD` : Kredensial login

3. Menjalankan MinIO

Jalankan container:

```
docker compose up -d
```

Verifikasi container berjalan:

```
docker ps
```

Jika berhasil, akses melalui browser:

`http://localhost:9001`

Login menggunakan:

- Username: `admin`
- Password: `admin123`

4. Pembuatan Bucket Medallion

Setelah login ke console MinIO:

1. Klik **Create Bucket**
2. Buat bucket dengan nama:
   - bronze
   - silver
   - gold

Struktur logical Data Lake sekarang menjadi:

```
MinIO Object Storage
|-- bronze/
|-- silver/
`-- gold/
```

Pada tahap ini, kita telah berhasil mengimplementasikan **Storage Layer** dalam arsitektur Big Data sederhana berbasis container.

3.5 Simulasi Ingestion Data ke Bronze Layer (Raw Data & Immutability)

Pada tahap ini, kita akan mensimulasikan proses *data ingestion* ke dalam zona **bronze**. Bronze layer berfungsi sebagai penyimpanan data mentah (*raw data*) yang bersifat **immutable**, artinya data tidak boleh dimodifikasi setelah disimpan.

1. Konsep Bronze Layer

Karakteristik utama Bronze Layer:

- Menyimpan data dalam format asli (CSV, JSON, log, dll.)
- Tidak dilakukan pembersihan atau transformasi
- Bersifat **append-only**
- Mendukung *data lineage*

Dalam praktik industri, bronze layer biasanya menerima data dari:

- Sistem transaksi (OLTP)
- API eksternal
- Streaming data
- Batch upload

Pada praktikum ini, ingestion dilakukan secara manual untuk mensimulasikan proses batch.

2. Menyiapkan Dataset Mentah

Masuk ke folder project:

```
cd ~/bigdata-lab-1
mkdir raw-data
cd raw-data
```

NOTE: Dataset sample.csv akan disediakan oleh asisten praktikum.
Buat file contoh `sample.csv`:

```
id,name,age,city
1,Andi,21,Bandar Lampung
2,Budi,23,Jakarta
3,Siti,,Surabaya
4,Rina,22,Bandung
```

File ini sengaja mengandung nilai kosong (missing value) untuk mensimulasikan kebutuhan pembersihan data pada layer berikutnya.

3. Mengunggah Data ke Bucket Bronze

Terdapat dua metode ingestion:

Metode A: Melalui MinIO Web Console

1. Buka `http://localhost:9001`
2. Login sebagai admin
3. Masuk ke bucket **bronze**
4. Klik **Upload**
5. Pilih file `sample.csv`

Metode B: Menggunakan MinIO Client (CLI) Masuk ke container MinIO:

```
docker exec -it bigdata-minio bash
```

(Opsional: jika menggunakan mc client)

4. Prinsip Immutability

Setelah file berada di bucket bronze:

- File tidak boleh diubah
- Jika ada pembaruan data, file baru harus diunggah
- Versi sebelumnya tetap dipertahankan

Sebagai praktik yang lebih baik, struktur folder dapat dibuat berbasis partisi waktu:

```
bronze/
|-- 2025/
|   |-- 07/
|   |-- 08/
|   ‘-- 09/
```

Contoh penyimpanan:

```
bronze/2025/07/sample_20250701.csv
```

Pendekatan ini memungkinkan:

- Audit data
- Reprocessing historis
- Debugging pipeline

5. Verifikasi Ingestion

Pastikan file berhasil diunggah dengan:

- Mengecek melalui Web Console
- Menggunakan S3 API (boto3) untuk membaca object

Pada tahap ini, proses ingestion telah selesai dan sistem telah memiliki sumber data mentah yang siap diproses ke Silver Layer.

3.6 Menambahkan Container Compute (Python) untuk Transformasi Data

Pada tahap ini, kita akan menambahkan container terpisah yang berfungsi sebagai **compute layer**. Container ini akan membaca data dari bucket *bronze*, melakukan transformasi sederhana, dan menyimpan hasilnya ke bucket *silver*.

Pendekatan ini mensimulasikan arsitektur seperti:

S3 + Spark atau Data Lake + ETL Engine

1. Menambahkan Service Python pada Docker Compose

Edit file `docker-compose.yml` dan tambahkan service berikut:

```
compute:
  image: python:3.11-slim
  container_name: bigdata-compute
  volumes:
    - ./app:/app
  working_dir: /app
  depends_on:
    - minio
  tty: true
```

Sehingga struktur akhir menjadi:

```
services:
  minio:
    ...
  compute:
    ...
```

Service ini akan menjalankan Python container yang terhubung ke jaringan internal Docker yang sama dengan MinIO.

2. Membuat Folder Aplikasi Compute

Buat folder aplikasi:

```
mkdir app
cd app
```

Buat file `requirements.txt`:

```
boto3
pandas
pyarrow
```

Buat file `transform.py`:

```
import boto3
import pandas as pd
from io import BytesIO

# Konfigurasi koneksi ke MinIO
s3 = boto3.client(
    's3',
    endpoint_url='http://minio:9000',
    aws_access_key_id='admin',
    aws_secret_access_key='admin123'
)

# Membaca file dari bucket bronze
response = s3.get_object(Bucket='bronze', Key='sample.csv')
data = response['Body'].read()

df = pd.read_csv(BytesIO(data))

# Transformasi sederhana (contoh)
df_clean = df.dropna()
df_clean['processed_at'] = pd.Timestamp.now()
```

```
# Simpan ke format Parquet (Silver Layer)
output_buffer = BytesIO()
df_clean.to_parquet(output_buffer, index=False)

s3.put_object(
    Bucket='silver',
    Key='sample_clean.parquet',
    Body=output_buffer.getvalue()
)

print("Transformasi selesai: Bronze ke Silver")
```

3. Install Dependency di Container Compute

Kembali ke root project, jalankan:

```
docker compose up -d
docker exec -it bigdata-compute bash
```

Di dalam container:

```
pip install -r requirements.txt
```

4. Simulasi Alur Data

Langkah simulasi:

1. Upload file `sample.csv` ke bucket **bronze** melalui MinIO Console.
2. Masuk ke container compute.
3. Jalankan:

   ```
   python transform.py
   ```

4. Periksa bucket **silver**.

Jika berhasil, file `sample_clean.parquet` akan muncul di bucket silver.

5. Makna Arsitektural

Dengan konfigurasi ini:

- MinIO berperan sebagai **Storage Layer**
- Container Python berperan sebagai **Compute Layer**

- Data berpindah dari Bronze ke Silver tanpa memodifikasi data asli

Konfigurasi ini mencerminkan prinsip fundamental sistem Big Data modern:

Storage terpisah dari Compute, namun terhubung melalui protokol standar (S3 API).

3.7 Transformasi Silver ke Gold Layer (Agregasi dan Optimasi Analitik)

Setelah data melalui proses pembersihan pada Silver Layer, tahap berikutnya adalah membangun **Gold Layer**. Gold Layer berisi data yang telah:

- Diagregasi
- Diringkas sesuai kebutuhan bisnis
- Dioptimalkan untuk analitik dan visualisasi

Layer ini merepresentasikan *business-ready dataset*.

1. Konsep Gold Layer

Karakteristik Gold Layer:

- Struktur data lebih sederhana
- Fokus pada kebutuhan analitik
- Minim transformasi ulang
- Siap dikonsumsi BI tools

Jika Silver bersifat *cleaned data*, maka Gold adalah *analytical data mart*.

2. Menambahkan Script Agregasi

Masuk ke folder `app/` dan buat file baru:

```
nano aggregate.py
```

Isi dengan kode berikut:

```
import boto3
import pandas as pd
from io import BytesIO

# Koneksi ke MinIO
s3 = boto3.client(
    's3',
    endpoint_url='http://minio:9000',
    aws_access_key_id='admin',
    aws_secret_access_key='admin123'
)

# Membaca file dari Silver
response = s3.get_object(Bucket='silver', Key='sample_clean.parquet')
data = response['Body'].read()

df = pd.read_parquet(BytesIO(data))

# Agregasi sederhana: rata-rata umur per kota
df_gold = (
    df.groupby('city')
      .agg(avg_age=('age', 'mean'),
           total_records=('id', 'count'))
      .reset_index()
)

df_gold['generated_at'] = pd.Timestamp.now()

# Simpan ke Gold Layer
output_buffer = BytesIO()
df_gold.to_parquet(output_buffer, index=False)

s3.put_object(
    Bucket='gold',
    Key='city_summary.parquet',
    Body=output_buffer.getvalue()
)

print("Transformasi selesai: Silver ke Gold")
```

3. Menjalankan Proses Agregasi

Masuk ke container compute:

```
docker exec -it bigdata-compute bash
```

Jalankan:

```
python aggregate.py
```

Jika berhasil, file `city_summary.parquet` akan muncul di bucket **gold**.

4. Alur Arsitektur Lengkap

Dengan tahap ini, alur data menjadi:

1. Raw data masuk ke **Bronze**
2. Data dibersihkan ke **Silver**
3. Data diagregasi ke **Gold**

Secara arsitektural:

- MinIO = Storage Layer
- Python Container = Compute Layer
- S3 API = Interface komunikasi

Pipeline ini merepresentasikan implementasi sederhana Medallion Architecture dalam sistem Big Data modern.

5. Interpretasi Hasil

Dataset pada Gold Layer kini:

- Lebih kecil ukurannya
- Lebih cepat di-query
- Siap digunakan untuk dashboard atau laporan

Setelah ini mahasiswa telah mengimplementasikan alur end-to-end Data Lake berbasis container, mulai dari ingestion hingga agregasi analitik.

4 Latihan

4.1 Latihan Terpandu: Implementasi Pipeline Menggunakan World Happiness Report

Pada bagian ini, akan diberikan contoh lengkap implementasi Medallion Architecture menggunakan dataset publik **World Happiness Report**.

Dataset dapat diunduh dari:

```
https://ourworldindata.org/happiness-and-life-satisfaction
```

File yang digunakan dalam praktikum adalah file CSV dengan struktur umum:

```
country,year,life_ladder,gdp_per_capita,
social_support,healthy_life_expectancy
```

Langkah 1: Upload ke Bronze

Unggah file CSV ke bucket **bronze** melalui MinIO Console.

Struktur:

```
bronze/
`-- happiness_raw.csv
```

Data pada layer ini bersifat immutable dan tidak boleh dimodifikasi.

Langkah 2: Transformasi ke Silver

Buat file `transform.py` dengan fungsi:

- Membaca CSV dari bronze
- Menghapus baris dengan nilai kosong
- Standarisasi nama kolom
- Menambahkan kolom `processed_at`
- Menyimpan dalam format Parquet ke silver

Contoh potongan transformasi:

```
df.columns = df.columns.str.lower().str.strip()
df_clean = df.dropna()
df_clean['processed_at'] = pd.Timestamp.now()
```

Output:

```
silver/
`-- happiness_clean.parquet
```

Langkah 3: Agregasi ke Gold

Buat file `aggregate.py` untuk menghasilkan ringkasan:

- Rata-rata happiness score per negara
- Rata-rata GDP per region

Contoh agregasi:

```
df_gold = (
    df.groupby('country')
      .agg(avg_happiness=('life_ladder','mean'),
           avg_gdp=('gdp_per_capita','mean'))
      .reset_index()
)
```

Simpan hasil ke bucket **gold**.
Output:

```
gold/
`-- country_summary.parquet
```

Analisis Hasil

Dataset pada Gold Layer kini:

- Lebih kecil ukurannya
- Siap digunakan untuk dashboard
- Sudah sesuai kebutuhan analitik

Pipeline Bronze -> Silver -> Gold telah berhasil diimplementasikan secara end-to-end.